Reber Grammar

Artificial grammar learning with transition memory

Reber's grammar (1967) is a small finite-state grammar over the letters T, P, S, X and V. Strings start with B and end with E. In artificial grammar learning experiments, participants who memorized strings generated by the grammar could later tell grammatical from ungrammatical strings, without being able to state the rules. The grammar has since become a standard benchmark for sequence learning in neural networks. Every letter labels two different transitions, so the next letter depends on the current state, not just on the letter before it.

This notebook implements the grammar as a finite automaton in a "transition" memory component. The query consists of two blocks, the current state and the current letter, and the memory returns the next state. The state is fed back and combined with the next letter. A start state input, tagged "priority", overrides the fed-back state and resets the automaton at the beginning of each string.

Each of the twelve rules is taught once. The automaton then classifies grammatical strings generated by the grammar and ungrammatical strings derived from them by substituting a single letter. A string is accepted if the automaton reaches the state "end" after its last letter. An undefined transition returns the empty set, and the automaton stays halted for the rest of the string.

Requirements

```
<< "Circuits.m"
```

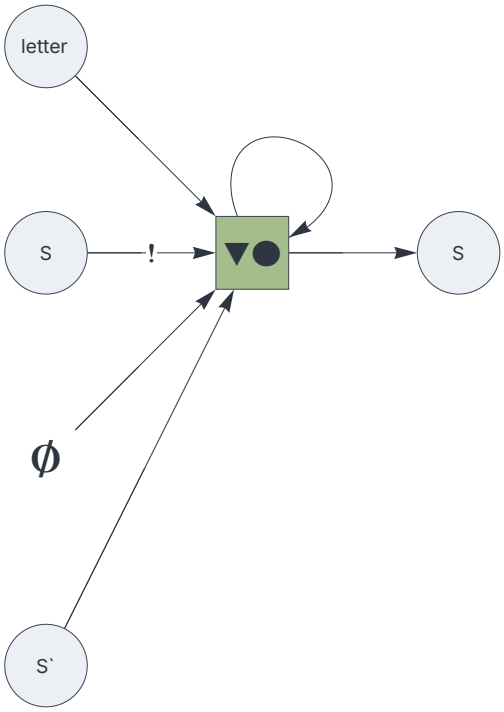
Circuit

```
jsonstr = Import[NotebookDirectory[] <> "/ReberGrammar.json", "String"];
```

```
jsonstr =
```

```
{ "hyperparameters": {"default": [2000, 10], "12": [2000, 0]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [1], "label": "S'
"},
    {"component": "input", "plugin": "codec", "send": [3], "label": "S"},
    {"component": "input", "plugin": "codec", "send": [4], "label": "letter"},
    {"component": "empty", "send": [2]},
    {"component": "transition", "send": [11, 12], "threshold": 1.0,
        "receive": [1, 2, [{"slot": 3, "tags": ["priority"]}, 11], 4]},
    {"component": "output", "plugin": "codec", "receive": [11], "label": "S"}
  ]}
```

```
circ = Circuit[jsonstr];
f = circ["function"];
circ["schematics"][ImageSize→270]
```



Dataset

Import the grammar:

```
grammar = Rest[Import[ NotebookDirectory[] <> "/ReberGrammar.csv"]];
```

```
begin    B    0
0        T    1
0        P    2
1        S    1
1        X    3
2        T    2
2        V    4
3        X    2
3        S    5
4        P    3
4        V    5
5        E    end
```

Train

Each rule once. The start state input carries the rule's state and, tagged "priority", overrides the fed-back state.

```
Scan[f[#[[3]], #[[1]], #[[2]]] &, grammar]
```

Test setup

Reference recognizer, used only to label the test strings.

```
table = Association[({#1, #2} → #3) & @@@ grammar];
grammaticalQ[s_String] :=
  Fold[Lookup[table, Key[{#1, #2}], Missing[]] &,
  "begin", Characters[s]] === "end";
```

Grammatical strings: random walk through the grammar.

```
generate[] := Module[{state = "begin", letters = {}, rule},
  While[state =!= "end",
    rule = RandomChoice[Select[grammar, First[#] === state &]];
    AppendTo[letters, rule[[2]]];
    state = rule[[3]];
  StringJoin[letters]]
```

Ungrammatical strings: substitute one inner letter until the string leaves the grammar.

```
randomSubstitution[s_String] :=
    With[{i = RandomInteger[{2, StringLength[s] - 1}]},
    StringReplacePart[s,
      RandomChoice[DeleteCases[Characters["TPSXV"],
      StringTake[s, {i}]]], {i, i}]];
  substitute[s_String] := NestWhile[randomSubstitution[s] &,
  s, grammaticalQ];
```

Run the automaton: the first letter comes with the start state, the remaining letters arrive alone.

```
trace[s_String] := Module[{letters = Characters[s], first},
  first = f[{}, "begin", First[letters]];
  Prepend[f[{}, {}, #] & /@ Rest[letters], first]];
acceptedQ[s_String] := Last[trace[s]] === "end";
```

Generate test data

```
positives = Table[generate[], 500];
negatives = substitute /@ positives;
```

Run

```
Do[Print[s, "   ",
    StringRiffle[MapThread[#1 <> ":" <>
    ToString[Replace[#2, Null → "-"]] &,
      {Characters[s], trace[s]}], " "],
    "   ", If[acceptedQ[s], "accepted", "rejected"]],
  {s, Join[Take[positives, 3], Take[negatives, 3]]}];

Print["grammatical accepted:   ",
    Count[positives, _?acceptedQ], "/500"];
Print["ungrammatical rejected: ",
    Count[negatives, _?(! acceptedQ[#] &)], "/500"];
```

```
BTSXSE   B:0 T:1 S:1 X:3 S:5 E:end   accepted
BTXSE   B:0 T:1 X:3 S:5 E:end   accepted
BPTTVVE   B:0 P:2 T:2 T:2 V:4 V:5 E:end   accepted
BTXXSE   B:0 T:1 X:3 X:2 S:- E:-   rejected
BVXSE   B:0 V:- X:- S:- E:-   rejected
BPVTVVE   B:0 P:2 V:4 T:- V:- V:- E:-   rejected
grammatical accepted:   500/500
ungrammatical rejected: 500/500
```